Interview Drill — creative-automation-pipeline

Skim, do not read aloud. Volunteer the GAPs before they find them. Lead with strengths, own the holes.

TOP 3 TO OWN FIRST

1. **Mid-run crash / ephemeral state.** In-memory Map + fire-and-forget IIFE. Crash loses every in-flight run; frontend gets 404. Deepest hole. Volunteer it.
2. **How compliance actually scores.** They will go line by line. Name the bugs yourself: empty-palette-fails-all, overlay pollutes color check, prohibited-words only scans your own message string.
3. **Concurrency / shared OUTPUT_DIR.** Two runs clobber the same files; /manifest returns the wrong run's data. Correctness bug, not just scale. Easy for them to spot.

SAY THIS, NOT THAT

DO NOT SAY	Say instead
"TWO AI AGENTS"	"Two bounded orchestration modules; deterministic for-loops wrapping tool calls. No reasoning loop."
"EVERY AI CALL LOGS TOKENS, COST, LATENCY"	"Per-call tokens + cost for Claude; run-level latency. Gemini image cost is stubbed at $0 today."
"RETRIES ON THE AI CALLS"	"Retries on Gemini image gen and Dropbox. Claude prompt + vision are not retried yet, known gap."
"MASTRA-COMPATIBLE, ONE-IMPORT SWAP"	"API-shape compatible so step contracts port; real Mastra adds resume/ persistence I don't have."
"COMPLIANCE-CHECKED CREATIVES OUT"	"Two deterministic gates (color, legal words) + one advisory vision check; here are the failure modes."

THE 10 QUESTIONS — COMPRESSED ANSWERS

1 Process restarts mid-run. What happens?

A: Total loss. State only in in-memory Map (runStore); workflow runs in fire-and-forget IIFE in the Express process; no queue, no checkpoint. Restart = empty Map, frontend polls /status and gets 404. Files on disk survive but nothing points to them. **GAP: SEVERE**

2 Step 4 fails on asset 30 of 100. What about 1 to 29?

A: Whole run fails. Every step re-throws on first item error; workflow returns error, no partial manifest. Assets 1-29 orphaned on disk. Zero per-item isolation in gather/render/overlay. Only ComplianceChecker isolates per-item. **GAP: SEVERE**

3 Walk me through retry/backoff. What is retryable and where applied?

A: base*2^(n-1) backoff, +/-20% jitter, 4 attempts. 429/5xx retryable, 4xx terminal, unknown defaults retryable. Applied to Gemini + Dropbox only. **Two real bugs:** terminal message hardcoded "Image generation failed" even for Dropbox; classification is string-regex so "500ms" could read as HTTP 500. **GAP: MINOR**

4 Image gen and logo check are non-deterministic. Acceptable in compliance?

A: Be precise. Color check + legal-word scan are deterministic (trust as gates). Gemini (no seed) and Claude vision (no temperature 0) are non-deterministic; logoPresent can flip between runs. Treat vision as advisory; pin temp 0 + reference-logo match before it gates. **GAP: MEDIUM**

5 No seed, no temperature. Can you reproduce a rejected image?

A: No. Persist the final PNG + manifest, but not the resolved prompt, seed, or model version. Fix: capture prompt + seed + model in organizeOutputs manifest. Small add. **GAP: MEDIUM**

6 Exactly how does ComplianceChecker score palette, logo, words? Where does each fail?

A: Spend time here.

- **PALETTE:** 50x50, quantize to 32-buckets, top 5 dominant, score = avg(1 - nearest RGB distance/max), pass ≥ 0.3. Fails: 0.3 is loose; runs on final.png so it scores its OWN black overlay bar; empty palette = 0 = every image FAILS.
- **LOGO:** Claude vision "any logo/text/watermark?" No reference-logo compare; campaign TEXT reads as logo, so reports compliant when real logo absent. No retry, errors swallowed to false.
- **WORDS:** lowercase substring includes() on campaignMessage ONLY. Not image, not product copy, not OCR. Substring not word-boundary = false positives.

GAP: THIS IS THE ONE THEY DRILL

7 Concurrency model? Two campaigns POST at once, what breaks?

A: Map keyed by runId is safe; filesystem is not. OUTPUT_DIR is one shared dir, so both runs write the same final.png / manifest.json / compliance.json / generated/. They clobber. /manifest + /compliance read the single file with no runId, so always return the LAST run. Logo is global too. Fix: namespace paths by runId.
GAP: SEVERE

8 What is the runtime doing during a 100-product run? Where's the parallelism?

A: None. Single Node process, all loops sequential, no Promise.all, no workers, no queue. 300 serial Sharp ops + 100 serial Claude+Gemini round trips + serial vision. Defensible for POC: correctness first, then bounded per-product concurrency + queue. **GAP: SCALE ONLY**

9 Manifest says uploadStatus:failed but run completes. Success or failure?

A: Soft failure by design. organizeOutputs catches per-file upload errors, stamps failed + falls back to local path, run still "done." Reasonable (degrade, don't fail) but frame as deliberate. Gap: top-level status doesn't surface partial upload failure. **GAP: MINOR**

10 POC to production. What do you build first?

A, IN ORDER: (1) durable run state + job queue (Redis/BullMQ or SQS + runs table in the Postgres stub) so crashes resume and /status survives. (2) Per-run output namespacing. (3) Per-item isolation + partial-success manifest. (4) Auth + multi-tenancy (zero auth today; loadFromUrl fetches any user URL = SSRF). (5) Harden compliance: temp 0, reference-logo match, OCR for prohibited words, real palette thresholds.
STRONGEST QUESTION, LEAD WITH QUEUE + RUNS TABLE

Accurate to lean on: 6 sequential steps, Zod input+output per step validated by the runner, 3 aspect ratios, SVG message + logo overlay, runId + 750ms polling (no SSE in code), pluggable storage (local/dropbox live), Postgres deferral is honest (db is an empty stub).